

CyberGear

高速化・安定化 改修説明書

電流モードを基盤とした軌道追従の再設計
人間・Codex共同開発向け

位置・速度・加速度の整合した参照を使い、
移動速度とフィードバック剛性を分離する。
修正範囲はCyberGear制御周辺に限定。

対象: catch2026_robstride / fix

基準コミット: 2a88ed1e31dda6a063dd62f70259de9111d552a

調査日: 2026-09-09

種別: 静的レビュー・改修仕様 / 実機未検証

第1部 技術説明・改修設計・検証手順

第2部 Codex向け作業指示

第3部 範囲外の不具合候補

付録 数式検算の概要

CyberGear 高速化・安定化 改修説明書

対象：catch2026_robstride / fix

基準コミット：2a888ed1e31dda6a063dd62f70259de9111d552a

調査日：2026-09-09 / 対象モーター：CyberGear ID 0x7f / ホスト：0xfe

本書はコードの静的レビューに基づく改修仕様である。ファームウェアの変更・書込み、実機試験、対象プロジェクトのビルドは実施していない。数値例は設計開始点であり、安全性・到達時間・安定性を実機で保証する値ではない。実装者は、作業開始時のHEADと本コミットとの差分を確認する。

1. 結論と実装順序

第1案は、現行の電流モードを維持し、「制約付き軌道生成+速度・加速度フィードフォワード+補償量を制限したESO」に変更すること。帯域を上げる前に、移動速度と閉ループ剛性を分離する。

現行コードは既に電流モード+外側ADRCである。古い運動制御モードの $K_p=8$, $K_d=4$ を前提にした単なるゲイン変更は、今回のコードには適用できない。[S1][S2]

実装は、まず保護・ログ・通信状態を整え、次に軌道と制御則を変更する。その後に速度上限を段階的に上げ、最後に必要な範囲だけ減衰比・制御帯域を調整する。内蔵位置モードの再採用、高帯域ESO、オンライン慣性同定、摩擦の強い不連続補償を同時投入しない。

運動制御モードの内蔵PDは比較・退避用に残す。ただし走行中の自動モード切替は実装しない。モード変更は明示的な停止・確認・再初期化を伴う手動操作にする。

対応範囲

区分	対象	方針
主な修正	Core/Src/cybergear.c, Core/Inc/cybergear.h	制御、プロトコル、状態管理、制限値を改修
新規追加	cybergear_controller.*, cybergear_trajectory.*, 必要なら cybergear_config.h	HAL非依存の計算部を分離
最小限の接続変更	main.c のCyberGear受信、起動、TIM6呼出し、専用ログ	RobStride側の意味・値・周期を維持
ビルド	新規ソース登録に必要なビルド設定のみ	実際のCMake構造を確認して追記
修正禁止	RobStride制御、幾何変換、上位CAN仕様、全体初期化の仕様、共有CAN設定の一括変更	候補は別紙に記録し、人間の承認を待つ

リポジトリ全体の整形、CubeMX再生成、共有FDCANの全キュー破棄、既存オフセットの推測修正は行わない。

2. 現行コードから分かったこと

2.1 実際の処理経路

初期化要求受信後、cybergear_base_init()、速度モードによる cybergear_homing()、RobStride初期化、cybergear_start_position_adrc() の順に進む。最後にTIM6を開始する。ADRC起動処理はSTOP、電流モード書込み、電流ゼロ書込み、ENABLEを行う。[S1][S2]

TIM6のベース周期は、コードの80 MHzクロック設定と Prescaler=79, Period=999 から1 msと計算される。しかしCyberGearを呼ぶのは10相のうちphase 0のみなので、**通常のCyberGear指令周期は10 ms = 100 Hz**。1 kHz制御ではない。RobStrideはphase 1/2/3、上位への角度送信は10相終了時である。[S2][S4]

UARTログはmainループで出力され、制御はTIM6割込みにある。「printfの出力時間がそのまま制御周期になる」という旧構造の診断は、この版には当てはまらない。ただし割込みマスク区間や同優先度ISRの長時間実行は別問題である。[S2][S4]

2.2 速度を制限している設定

設定	現行値	評価
CYBERGEAR_ADRC_REFERENCE_RAD_S	0.4 rad/s	約22.9 deg/s。指令軌道そのものが遅い
CYBERGEAR_ADRC_CONTROL_RAD_S	4 rad/s	Hzではない。角周波数の設計パラメータ
CYBERGEAR_ADRC_DAMPING_RATIO	2.0	公称モデルでは強い過減衰
CYBERGEAR_ADRC_OBSERVER_RAD_S	10 rad/s	位置から速度・外乱を推定
CYBERGEAR_ADRC_B0_RAD_S2_PER_A	10	入力ゲインの仮定。実測根拠はコードから確認できない
CYBERGEAR_ADRC_CURRENT_LIMIT_A	10 A	ソフト上限。連続許容電流や安全電流とは同義でない
CYBERGEAR_ADRC_CURRENT_SLEW_A_S	5 A/s	10 msで最大0.05 Aしか変化しない
フィードバック失効／更新間隔上限	100 ms／50 ms	高速化時には停止余裕から再評価が必要

これらは定義と実装を直接確認した値である。[S1]

0.4 rad/sのランプ指令は、90度移動で最終目標に達するだけでも約3.93秒かかる。これは実機の厳密な最短到達時間ではなく、**内部参照が最終値に達する時間**である。追従遅れや停止判定はさらに別に存在する。

2.3 構造的な遅さ：動く目標に対して速度ゼロを追っている

現行制御則は概ね次式である。 z_1, z_2, z_3 はESOによる位置・速度・総外乱加速度の推定値、 r はランプ化した位置指令である。[S1]

$$i_{\text{req}} = \{ \omega_c^2 (r - z_1) - 2 \zeta \omega_c z_2 - z_3 \} / b_0$$

r は移動する一方、速度項には r_{dot} がない。したがって移動中も $-2 \zeta \omega_c z_2$ が速度を抑える。完全な外乱補償・正しい公称モデル・非飽和・十分長い等速区間を仮定すると、

$$q_{\text{ddot}} + 2 \zeta \omega_c q_{\text{dot}} + \omega_c^2 q = \omega_c^2 r$$

定常ランプ追従誤差 $e = r - q = (2 \zeta / \omega_c) r_{\text{dot}}$

現行値では係数 $2 \text{ zeta} / \omega_c = 1 \text{ s}$ 。 $r_{\text{dot}}=0.4 \text{ rad/s}$ なら **0.4 rad、約22.9度の遅れ**が公称モデルだけでも生じる。実機の観測値ではないが、単純に参照速度を上げると追従誤差まで増える理由を説明する。

また、公称閉ループ極は $s^2+16s+16=0$ の根、約 $-1.072, -14.928 \text{ [1/s]}$ 。遅い極の時定数は約0.93秒である。安定化のために減衰を増やした結果、停止後の収束も遅くなっている。

2.4 発振について確定できる範囲

ログ・慣性・機械共振・実電流がないため、実機発振の単一原因は断定できない。ただしコード上では、電流スルー制限、低帯域の推定速度、未同定の b_0 、サンプル／通信遅延、誤差依存の外乱リークが関与し得る。

電流スルー5 A/sでは、+5 Aから-5 Aへの反転に最短2秒、+10 Aから-10 Aなら4秒かかる。実際にこの電流域を使っているかはログで確認する必要があるが、制動が必要なときに正方向電流が長く残る構造になり得る。

一方、**現行の離散ESOゲインは誤りと決めつけない**。リークを除いた係数は、ZOHモデルのcurrent observerに対する既知の離散極配置と一致する。また、現行コードは制限後の電流を次の予測に戻しており、この点は維持すべきである。[S1][S5]

3. 制御モードの整理

本プロジェクトの列挙値と指令APIは以下である。表はコードと公開実装の動作モデルに基づく。搭載ファームウェアの版・内部ゲイン・各制限レジスタの適用範囲は別途確認する。[S1][S3][S6]

run_mode	名称	主な指令	どこで制御するか／今回の扱い
0	運動制御／Operation、MIT-style	通信type 1で位置・速度・Kp・Kd・トルクFF	内蔵PDを利用。比較・回避用
1	位置制御	loc_ref, 速度・電流制限等	内部位置ループ。過去の発振経験から初回改修では不採用
2	速度制御	spd_ref	内部速度ループ。現行ホーミングで使用。通常制御への流用は別評価
3	電流制御	iq_ref [A]	内部電流ループを残し、位置／速度ループをSTM32で実装。第1案

3.1 運動制御モード

設計上のモデルは次式である。機械座標、トルクの単位・換算、ファームウェアの詳細は実機資料と照合する。

$$\tau_{\text{cmd}} = K_p (q_{\text{ref}} - q) + K_d (v_{\text{ref}} - v) + \tau_{\text{ff}}$$

このモードの velocity は「速度上限」ではなく、PD則の速度目標である。位置が移動するのに velocity=0 を固定すると減衰項が移動を妨げる。運動制御案でも必ず軌道の $q_{\text{ref}}, v_{\text{ref}}$ を対で渡す。

利点は、位置・速度フィードバックを使う内側PDをモーター側に任せられ、外部CANの遅延に外側の全PDを載せずに済むこと。欠点は、内部実装・ゲイン上限に拘束され、静摩擦や負荷に対する偏差が残る得ること。外部ESOのトルク補償まで同時投入せず、まずPD+軌道だけを比較する。

3.2 電流モード

電流モードはPWMの直接駆動ではない。内部電流ループは残る。外側は概ね、

```
J(q, payload) q_ddot = Kt_out iq - tau_load - tau_friction
q_ddot = b0 iq + f,    b0 ≈ Kt_out / J
```

として扱う。慣性変化は総外乱に含められるが、ADRCが任意の慣性変化・遅延・飽和を無条件に吸収できるわけではない。b0 の符号・範囲、制御遅延、制動能力の確認は必要である。[S5]

3.3 feedback.mode と run_mode を混同しない

現行 feedback.mode はCAN ID上位の2ビットから取り出す運転状態で、公開実装では0=Reset、1=Calibration、2=Runとして解釈される。**feedback.mode==2 は速度モードの証明でも電流モードの証明でもない**。[S1][S3][S6]

現行 cybergear_set_run_mode() は書き込みのキュー投入成功後にソフト側 motor->run_mode を更新する。その値を読むだけでは実機モードの確認にならない。起動時には 0x7005 のreadbackを追加する。

制御モードを変更するときは、制御所有権を状態機械に一本化し、STOP／停止確認、モード書き込み／readback、ゼロ指令、ENABLE／Run確認、現在位置での軌道・ESO初期化という順に進める。位置・速度ゼロ指令や現在位置指令の順序はモード別に定義する。停止確認と機械的静止は異なる。

4. 推奨制御系

4.1 構成

```
上位位置目標 target_angle[0]
-> 値・機械可動域の検証
-> 連続軌道 qd, vd, ad (速度・加速度・jerk制限)
-> 2自由度PD+加速度FF+制限付き外乱補償
-> 電流振幅・変化率の制限
-> CyberGear iq_ref

位置feedback -> タイムスタンプ付きESO -> z1, z2, z3
制限後・送信状況を反映した電流履歴 -----> ESO予測
```

主経路は位置測定のみで成立させる。既存フィードバックの速度値は、まず推定速度の検証・異常検出用にログへ残す。単純な位置差分へD項を置き換えない。±12.5 radを16ビットに変換する現行プロトコルでは、位置1カウントは約0.0003815 radであり、差分は周期を短くするほど量子化を増幅する。[S1]

4.2 制御則

```
kp_acc = wc^2
kd_acc = 2 zeta wc

i_track = { ad + kp_acc (qd - z1) + kd_acc (vd - z2) } / b0
i_dist  = bounded_and_rate_limited( -gamma z3 / b0 )
i_req   = i_track + i_dist
i_amp   = clamp(i_req, -I_max, +I_max)
i_cmd   = clamp(i_amp, i_prev - S_down dt, i_prev + S_up dt)
```

gamma は外乱補償の有効度で0～1。起動直後は0で推定状態を整え、設定時間で滑らかに上げる。通常運転中に目標変更のたびにゼロへ戻してはならない。

ad/b0 が加速に必要な電流を先回りして供給し、kd_acc * vd/b0 が移動中の不要な制動を相殺する。理想条件では追従誤差は次式になり、ゲインを上げずに移動参照へ追従できる。

$$e = qd - q$$

$$e_{ddot} + 2 \zeta \omega_c e_{dot} + \omega_c^2 e = 0$$

これは公称モデル上の性質であり、電流飽和・遅延・モデル不一致があれば誤差は残る。高速化はまず vd , ad の設計で行い、 ω_c の増加を第一手にしない。

4.3 軌道生成：小さく実装し、制約を検証する

実機時間と開発範囲を考慮し、初版は**制約検証付きの五次多項式軌道**を推奨する。最短時間のS字軌道ではないが、位置・速度・加速度を整合した形で生成でき、Cで単体試験しやすい。限界速度がなお不足する場合に、別コミットで検証済みのjerk制限S字生成器へ置換する。

静止から静止への距離 $D=|q_f - q_0|$ の移動では、 $s=t/T$ として、

$$q(t) = q_0 + (q_f - q_0) (10s^3 - 15s^4 + 6s^5)$$

$$T \geq \max(1.875 D/V_{\max}, \sqrt[3]{(10 \sqrt[3]{3}/3) D/A_{\max}}, \sqrt[3]{60 D/J_{\max}})$$

この静止端点条件では、上のTで速度・加速度・jerkの最大値を解析的に抑えられる。例えば90度、 $V_{\max}=1.5 \text{ rad/s}$, $A_{\max}=3 \text{ rad/s}^2$, $J_{\max}=15 \text{ rad/s}^3$ なら約1.96秒の参照軌道となる。現行ランプ約3.93秒との比較は**軌道時間のみ**であり、実機到達時間の予測ではない。

走行中に目標が変わる場合は、直前に生成した qd, vd, ad を新軌道の初期条件に使う。毎回実測位置へ参照を戻す、速度をゼロへ飛ばす、同一目標フレームごとに軌道を再始動する、といった実装は禁止する。

任意初期状態では、終端 $v_f=a_f=0$ の五次式を次で生成できる。

$$q(t) = q_0 + v_0 t + (a_0/2)t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5$$

$$Dq = q_f - q_0$$

$$c_3 = 10 Dq/T^3 - 6 v_0/T^2 - 1.5 a_0/T$$

$$c_4 = -15 Dq/T^4 + 8 v_0/T^3 + 1.5 a_0/T^2$$

$$c_5 = 6 Dq/T^5 - 3 v_0/T^4 - 0.5 a_0/T^3$$

この場合、静止端点用のTの公式だけでは制約を保証できない。0～T内の位置・速度・加速度・jerkの極値を検証し、実行可能なTを探索する。速度の極値は加速度の実根、加速度の極値はjerkの実根、jerkの極値はsnapの実根で求める。可動域検証には速度の実根も必要になる。端点も必ず評価する。T増加だけで必ず解けると仮定せず、反復回数と最大時間を設け、不可なら制動状態に入る。

探索はISRで無制限に行わない。候補生成・検証を回数制限付きで実行し、新軌道への切替時点の初期条件と一致させる。main側で非同期生成する場合、作成開始時の状態と有効化時の状態のずれを処理する。タイム内は有効な係数の評価を基本とする。

4.4 電流制限と軌道制約は別物

機械的な滑らかさは主に $A_{\max}, J_{\max}$ で制御する。電流スルーを極端に遅くして代用すると、誤差修正・制動も遅くなる。

ただし5 A/sを即座に大きくするのも避ける。まずログで電流スルー飽和率と符号反転時間を確認する。軌道FFだけの電流変化率は概ね $J_{\max}/b_0$ であり、 $J_{\max}=15, b_0=10$ なら1.5 A/s。滑らかな軌道なら、現行スルー値のままでもFF部分は追従可能である。残りにPD・外乱補償の余裕が必要になる。

スルー飽和が律速と確認できた場合のみ、例えば5→10→20 A/sと段階変更する。これは試験候補であり推奨定格ではない。通常制動で負方向の電流を作る際にも制限が働くことをテストする。非常停止は通常の軌道スルーを経由させず、故障方針に従って駆動停止または外部安全機構を使う。

5. ESOの扱い：既存の正しい部分を残す

5.1 離散モデル

外乱加速度を1ステップ内で一定とするモデルは、

```
x = [q, v, f]^T
A(T) = [[1, T, T^2/2], [0, 1, T], [0, 0, 1]]
B(T) = b0 [T^2/2, T, 0]^T
C = [1, 0, 0]
x_pred = A x_prev + B u_prev
e_y = q_meas - C x_pred
x_new = x_pred + L e_y

p = exp(-w0 T)
L1 = 1 - p^3
L2 = 3 (1-p)^2 (1+p)/(2T)
L3 = (1-p)^3/T^2
```

である。現行のリークなし部分はこの形式に一致する。[S1][S5]

予測入力には、振幅制限・スルー制限後の出力を用いる。TX失敗時に未送信の i_{req} を印加入力扱いしない。HAL成功はキュー投入成功にすぎないので、`last_queued_current` と `applied_current_estimate` を区別し、後者には有効性フラグを持たせる。バス異常時はモデルの入力が不明になるため、ESOで帳尻を合わせず停止する。

5.2 時刻と受信シーケンス

追加状態は、少なくとも `rx_sequence`, `rx_timestamp`, `observer_timestamp`, `last_control_timestamp`, `last_queued_timestamp`。受信時刻が同一msでも別フレームを識別できるよう、鮮度判定にタイムスタンプだけを使わない。

測定補正は新しいフレームに対して1回だけ行う。古い測定へ何度も補正を掛けない。制御周期と受信周期を別の時計として扱う。待ち時間を50 msで切っても、通常周期が5/10 msなら遅延の影響はその前から出る。

推奨実装は、補正済み状態をその受信時刻で保持し、新測定までの時間を、区間ごとの制限後電流履歴で予測して補正する。その後、制御時刻まで予測したコピーを制御に用いる。これにより未来時刻の推定へ古い測定を現在値として混ぜることを避ける。通信遅延により受信時刻とモーター内の測定時刻は一致しないため、残留遅延は別に計測・評価する。

初版でここまでの時刻整合を省く場合は、100 Hz固定周期・十分小さい受信遅延の条件を明記し、制御時刻へ直近測定を近似配置する。これは近似であり、非同期の可変dtを都合よく混在させない。200 Hz化は遅延分布確認後とする。

5.3 外乱リークと補償量

現行は追従誤差に応じて0.5~0.03 /sのリークを入れる。これは外乱蓄積を抑える一方、保持に必要な補償まで忘れ、定常偏差や非線形挙動の原因になり得る。リークを「必ず悪いバグ」とは扱わない。[S1]

初回A/Bではリークを残したまま軌道・FFのみを変更する。その後の別コミットで、通常運転の誤差依存リークを固定の小リーク、またはリークなし+外乱補償上限へ置き換える。補償上限を0.5~1 A等の小さい試験値から始め、必要な保持電流に応じて再評価する。これらも実機許容電流内という条件付きの例である。

長時間の電流飽和・高追従誤差・進行不足は「もっと推定で補う」条件ではなく、拘束・衝突・能力不足の疑いとして停止判定にする。単なる電流飽和でESOを無条件に凍結しない。総外乱には負荷だけでなくモデル誤差も含まれるためである。制限後入力の帰還、補償上限、モデル破綻検出を分けて実装する。

w_0 を上げれば常に堅牢になるわけではない。位置量子化・通信遅延・機械共振とトレードオフになる。初版の候補は現行10 rad/sを中心に8~12 rad/s程度とし、効果と入力ノイズを比較する。論文の一般的な帯域比を機械的に適用して3~10倍へ上げない。[S5]

6. パラメータ変更計画

値は機体・負荷に応じて確定する。表を「そのまま書き込む最適値」と解釈しない。

段階	変更	維持・確認するもの
A0	保護、ログ、readback、テストの追加	現行アルゴリズム・100 Hz・電流制限
A1	軌道のqd/vd/ad、2自由度制御を追加	$w_c=4$, $\zeta=2$, $w_0=10$ をまず維持。低い速度で比較
A2	V_{max} を0.4→0.8→1.2→1.5 rad/s等へ段階増加	A_{max}/J_{max} 、実電流・符号反転、機械余裕を監視
A3	ζ を2→1.2→1.0へ変更	w_c は固定。0.8前後は十分な余裕確認後の速度優先候補
A4	必要なら w_c を4→6→8 rad/s等へ	b_0 範囲・遅延・共振の試験に合格した範囲のみ
A5	外乱リーク方式、200 Hz化、S字軌道への変更	それぞれ別変更。効果がなければ戻す

速度優先でも、位置・速度許容差を極端に小さくして長い整定待ちを作らない。評価用の仮値として位置 $\pm 0.02 \sim 0.04$ rad、速度0.1~0.2 rad/s以下、50~100 msの連続成立などを置けるが、**既存上位シーケンスの完了条件を勝手に変更しない**。これは比較ログの指標とし、機械干渉・作業精度から人間が確定する。

制御帯域と慣性の関係も確認する。ESOなしの公称PDで実入力ゲイン比を $\rho = b/b_0$ とすると、実際の自然角周波数は $\sqrt{\rho} w_c$ 、減衰比は $\sqrt{\rho} \zeta$ になる。重い姿勢で b が小さくなると遅く、減衰も弱くなる。これは同定範囲を必要とする理由であり、慣性の軽い状態だけで決めない。

7. 周期・CAN・状態機械

7.1 200 Hz化する場合の最小変更

TIM6の1 ms自体は変更しない。CyberGearだけphase 0と5に置き、RobStrideのphase 1/2/3、上位角度送信phase 9をそのまま残す。初版は100 Hzのまま比較可能にする。[S2]

```

phase 0: CyberGear
phase 1: Right RobStride (変更なし)
phase 2: Left RobStride (変更なし)
phase 3: EL05 (変更なし)
phase 5: CyberGear (200 Hz設定時のみ)
phase 9: 上位角度送信 (変更なし)
```

コード設定からモーターバスはnominal 1 Mbit/sと計算される。拡張Classic CAN 8 byteを余裕込み160 bit/frame、指令1件に応答1件と仮定すると、現状4台各100 Hzで約13%、CyberGearのみ200 Hzなら約16%となる。追加読出し、再送、非同期フィードバック、他ノードの交通を含まない概算であり、アナライザ計測を優先する。[S2][S4]

500 Hzを初回から要求しない。平均バス負荷より、FIFO瞬間詰まり、ISR実行時間、応答の実周期、HAL再入の方が制約になる場合がある。

7.2 必須の状態機械

```
DISARMED
-> STOP_REQUEST / STOP_CONFIRMED
-> MODE_WRITE / MODE_READBACK
-> ZERO_COMMAND / ENABLE_WAIT
-> OBSERVER_WARMUP
-> RUN
-> CONTROLLED_STOP または FAULT_LATCHED
```

別に HOMING の所有者を定義する。ホーミング中は速度指令、通常運転中は電流指令、故障中は停止処理のみを許可し、mainとTIM6が相反する指令を出さない。全状態遷移には期限を設ける。

停止要求が成立したこと、STOPがCANへ送られたこと、モーターがReset状態であること、機械軸が静止したことは4つの別状態である。これを一つのboolで代用しない。

TX失敗時は故障理由をラッチし、通常指令を禁止する。停止フレームは期限・回数を制限して再試行する。現行もactive=false後の周期呼出しでSTOPを再発行し得るため、「停止を一度しか試みない」とは断定しない。問題は明示的な停止到達管理・理由・期限・代替安全動作がないことにある。[S1][S2]

バス断・bus-off時はSTOP送信で停止を保証できない。 またSTOP/iq=0は能動制動・機械的停止を保証しない。電流モードで最後の指令が保持される条件や内蔵通信タイムアウトは、搭載FWで確認する。未確認のウォッチドッグレジスタをCodexに創作させない。ソフト改修外でも、外部非常停止、機械的退避余裕、必要なら制動機構を実機試験の前提にする。

7.3 受信ディスパッチの注意

現行FDCAN3コールバックは、共通条件で通信type 2以外を除外した後にCyberGearパーサを呼ぶ。このままCyberGear側にtype 17パラメータ応答やtype 21故障応答を追加しても届かない。[S2]

共通の拡張ID・DATA・8byte検証を通した後、CyberGearの自ID・宛先を確認する専用ディスパッチへ渡し、そこで2/17/21を処理する。その後に既存RobStride用のtype 2制約と処理をそのまま残す。未知フレームをCyberGear用として誤消費しない。

8. 機械的制約とホーミング

±12.5 radは通信の表現範囲であり、このロボットの安全可動域ではない。ソフト可動域 q_{min}, q_{max} 、外側保護限界、速度・加速度・電流・温度制約を機体担当者が決める。角度の安易な±pi折返しは、ケーブルや有限可動域がある基部で別経路への回転を生むため禁止する。

制動余裕には検出遅延と電流反転の遅れを含める。単純な $v^2/(2a)$ だけで停止位置を見積もらない。遅延中に外向き加速度 a_{out} が続く最悪条件の概算は、

```
v_after = |v| + a_out t_delay
d_margin >= |v| t_delay + 0.5 a_out t_delay^2
           + v_after^2/(2 a_brake) + uncertainty_margin
```

である。ただし電源・CAN喪失時に能動制動が使えない場合、この式の a_{brake} を通常時の値で流用してはならない。

ホーミングは速度モードでセンサ端を両方向に測定し、平均位置へ移動した後、STOP→10 ms→SET_ZEROとする実装である。コメントに静止／整定確認をしないと明記されており、これは意図の可能性がある。[S2]

高速通常制御の改修と同時に、原点の定義や端検出の幾何を変更しない。ただし原点設定後に新しい座標のフィードバックを待つ、現在位置で軌道とESOをリセットする、残留速度がある場合に通常制御へ入らない、という境界の安全確認はCyberGearの起動仕様に含める。静止判定を加えることでサイクル時間が変わる点は人間レビュー事項とする。

9. 実機時間を節約する検証計画

9.1 PC側で先に終える検証

HAL非依存の制御・軌道計算部をホストビルドできるようにする。既存ファームウェアのテストと、本書添付の数式確認スクリプトは別物である。

検証	注入条件・確認内容
軌道	正負、短距離、ゼロ距離、反転、走行中目標変更、同一目標反復。位置／速度／加速度の連続性と極値制約
ESO	離散係数、一定入力、入力制限、測定重複、フレーム欠落、異なる時刻、再初期化
入力	NaN/Inf、範囲外、符号、過大dt、tick wrap、古い目標
CAN	モード3・iq_refのbyte列、type 2/17/21、別motor/host、DLC不一致、TX FIFO満杯
状態	STOP失敗、モードreadback不一致、enable失敗、起動feedback欠落、再起動、故障後の自動再arm禁止
スケジューラ	1000 tickでCG=100または200回、各RS=100回、上位=100回。順序と角度変換が不変
数値モデル	b/b0=0.25,0.5,1,2,4、遅延0/5/10/20 ms、摩擦・負荷段差・ノイズ・拘束・柔軟2慣性モデル

上記の比率は保証範囲ではなく、弱点を探すためのストレス試験範囲。合格領域を示し、不合格領域を隠さない。剛体モデルだけで実機安定性を宣言しない。乱数seedを固定し、実験条件と失敗ログを保存する。

ゴールデンベクトル例（ID 0x7f、host 0xfe、既存エンコード規約）：

```
run_mode=3 : ID 0x1200fe7f, data 05 70 00 00 03 00 00 00
iq_ref=1 A: ID 0x1200fe7f, data 06 70 00 00 00 00 80 3f
STOP      : ID 0x0400fe7f, data 00 00 00 00 00 00 00 00
```

これらは既存ソースから導くエンコード確認値であり、実機で受理されたという意味ではない。[S1][S3]

9.2 実機試験は4区分に絞る

最初は動作域を確保し、衝突・落下・挟まれを避けられる支持状態で実施する。停止不能や暴走を検出してから人手だけで止める試験計画にしない。

通信・符号確認。 モードreadback、0電流、位置と速度の符号、正負の小電流に対する動作を短く確認する。未知の大電流パルスを入れない。ログ・停止経路を先に検証する。

入力ゲインの概算。 同程度の位置・速度条件で許容範囲内の異なる電流に対する加速度を比較し、 $b \approx \Delta \text{acc} / \Delta i$ を推定する。差分ノイズを避けるため短区間の速度直線当てはめ等を使う。位置のみを使う場合は局所二次当てはめでもよい。静摩擦で全く動かない区間は同定に使わない。代表的な軽い／重い姿勢の2条件以上を残す。

同じ動作でA/B。 低速度で旧制御と軌道FF改修版を比較し、その後、速度だけを上げる。正方向・負方向・反転を含め、原点復帰と他軸の動作条件を合わせる。

最悪負荷と異常。 許容した最高速での停止、重い姿勢、目標反転を確認する。通信断・拘束の挙動は、まずモック／ベンチと低エネルギー条件で評価する。通常の高速度・大負荷でいきなり断線試験をしない。

9.3 ログと合否

制御周期でリングバッファへ数値を記録し、mainで後送または低頻度排出する。ISR内printfは禁止。バッファ満杯時はログを捨ててdrop数を数え、制御を待たせない。

```
t_us, state, fault_reason, target, qd, vd, ad, q_meas, v_feedback,
z1, z2, z3, i_track, i_dist, i_req, i_cmd,
amp_limited, slew_limited, rx_seq, rx_age_us, control_dt_us,
tx_queued, tx_failed, tx_fifo_free, busoff, tec, rec
```

速度だけでなく、初回許容帯到達時間、50～100 ms滞在までの時間、最大オーバーシュート、移動中追従誤差、電流の最大／RMS、スルー飽和率、符号反転時間、振動の減衰、更新周期最大値を比較する。

合否は、承認済みの可動域・電流・温度・遅延制約を守ること、目標固定後の振動が成長しないこと、反転時に制動が遅れ続けないこと、故障時に新規通常指令が出ないことを先に置く。その上で到達時間を短くする。小さいオーバーシュートは許容しても、増大する振動や境界越えを許容しない。

10. 人間とCodexの分担

人間は、機械可動域、電流・速度・温度の許容値、原点仕様、実機試験の安全条件、作業完了判定を確定する。Codexは、指定範囲の実装、ホストテスト、回帰テスト、差分一覧、未検証事項を担当する。

各コミットは、C1=観測と保護、C2=軌道、C3=2自由度制御、C4=ESO補償制限、C5=任意の200 Hz化／比較モード、と分ける。上位仕様やRobStrideへ影響が必要なら、その時点で変更を止め、別紙の未承認項目として提示する。

別紙 02_Codex作業指示.md を実装作業の入口とする。03_範囲外の不具合候補.md は情報共有用であり、まとめて修正する指示ではない。

11. 根拠・参照資料

ソースについてはすべて基準コミットの関数内容を確認した。行番号は後続変更で動くため、関数名とコミットを主な識別子にする。

- [S1] 対象リポジトリ Core/Src/cybergear.c : ADRC定数、cybergear_start_position_adrc, cybergear_control_position_adrc, 送受信・モードAPI。 [ソースを開く](#)
- [S2] 同 Core/Src/main.c : 初期化、ホーミング、FDCAN受信、TIM6、ログ、全体監視。 [ソースを開く](#)
- [S3] 同 Core/Inc/cybergear.h : モード・パラメータ・状態定義。 [ソースを開く](#)
- [S4] 同 Core/Src/can_init.c, Core/Src/stm32g4xx_hal_msp.c : 送信形式、フィルタ、CANクロック源、割込み優先度。 [ソースを開く](#)
- [S5] Gernot Herbst, “Practical Active Disturbance Rejection Control: Bumpless Transfer, Rate Limitation and Incremental Algorithm”, DOI: 10.1109/TIE.2015.2499168, 著者稿 arXiv:1908.04610。離散ESO式(6)～(10)、入力制限・初期化の議論を参照。本書のロボット固有の制御・保護仕様と数値例は本レビューの設計提案であり、論文による実機保証ではない。 [著者稿を開く](#)
- [S6] cybergear-robotics/cybergear, cybergear.c : 公開ドライバのコマンドエンコードと運転状態解釈。メーカー公式FW仕様ではなく、故障処理の未検証項目がREADMEに明記されているため、そのまま複製しない。 [ソースを開く](#)

本書では、メーカー原本マニュアルの当該FW版までの一致は確認できていない。したがって、内部ゲイン、連続許容電流、通信ウォッチドッグ、各制限パラメータが電流モードでも効くかは未確認項目として残す。

Codex作業指示：CyberGear制御に限定した改修

対象：T-semi-Tohoku-Uni/catch2026_robstride、基準ブランチ fix

基準コミット：2a888ed1e31dda6a063dd62f70259de9111d552a

そのまま渡せる指示

このリポジトリのCyberGear制御を、添付 01_改修説明書.md に従って高速化・安定化してください。現在は電流モード+ADRCです。古い運動制御PD版を前提にしないでください。目的は、有限のオーバーシュートを許容して移動を速くし、実機の試行錯誤を減らすことです。安全可動域を越えるオーバーシュートや持続／成長する発振は許容しません。

最初の応答では実装方針と変更ファイルを示し、その後は下記の小さい変更単位で進めてください。基準との差分や人間の未コミット変更を上書きしないこと。

0. 作業開始時の確認

git status とHEADを記録し、基準コミットとの差分を確認する。既存のローカル変更を保持し、担当者が並行して変更したコードを戻さない。必要なCMake設定・ツールチェーン・テスト環境を読んで、実際に使えるビルドコマンドを決める。説明書にないバグを見つけても範囲外なら修正せずメモへ追記する。

1. 変更範囲を限定する

変更可能なのはCyberGearのドライバ・外側制御・軌道生成・そのテストとログ。main.c はCyberGear用の起動、受信ディスパッチ、タイマ呼出し、専用診断の接続箇所だけを編集する。新規モジュールをビルドに追加する最小変更は許可する。

次は変更しない：RobStride制御、ID 3/4/5の値・周期・モード、target_angle[1..3] の解釈、既存角度オフセット、上位CAN ID/DLC/endian、全体の初期化・return仕様、共有CAN速度・自動再送・IRQ優先度、GPIO割当、全体reset処理。全体整形やCubeMX再生成もしない。

CyberGearの入力は従来どおり target_angle[0]。返す角度の座標系・原点定義を変えない。範囲外NaN等へのCyberGear側保護は追加するが、他軸の受信処理を書き換えてはならない。

2. C1：観測・保護・状態管理

制御式と通常の100 Hzを維持し、以下を追加する。

- 故障理由、rx_sequence、受信時刻、制御時刻、TX成功／失敗、FIFO空き、更新時間・データ鮮度、電流振幅／スルー制限のログ。
- HAL_OK を実機到達済みとみなさない送信状態。STOP要求、送信、Reset確認、機械的静止を区別する。
- CyberGear専用状態機械と送信所有権。FAULT後の通常指令・自動再armを禁止し、期限付きSTOP再試行を行う。
- 0x7005 のreadback。feedback.mode==2 をrun_modeの確認と誤認しない。
- type 17/21が、main.cの既存type 2限定フィルタより前でCyberGearに振り分けられる接続。RobStrideへの従来条件は保存する。
- NaN/Inf、機械範囲、過大dt、位置飛び、拘束疑い、温度、速度、通信失効の保護。未承認の機械上限値を適当に確定しない。

継続運転中の一般readbackや診断フレームは帯域・送信スロットを管理する。STOPと通常電流フレームを同時に大量投入しない。共有FDCANの全キュー破棄は他軸を巻き込むので禁止する。

3. C2：HAL非依存の軌道生成

標準は説明書の制約付き五次軌道とする。静止端点用の解析的T、任意初期qd/vd/adからの再計画、全区間の位置・速度・加速度・jerk制約の検証を実装する。同一目標の反復受信では軌道をリセットしない。走行中の目標変更でq/v/aを不連続にしない。

再計画の反復回数と実行時間に上限を設ける。ISRで未制限の根探索・多項式計画をしない。非同期計画では開始状態と切替状態の時刻の一致を保証する。実行可能な軌道が見つからない場合は直前軌道を無期限に走らせず、定義済みの制動状態へ移る。

4. C3：2自由度制御

次の式を採用する。最初は $w_c=4$ 、 $\zeta=2$ 、 $w_o=10$ を維持し、軌道とフィードフォワードの効果を切り分ける。電流上限・ソフト可動域は人間の承認値を使う。

```
i_track = (ad + wc*wc*(qd-z1) + 2*zeta*wc*(vd-z2)) / b0
i_dist = bounded_rate_limited(-gamma*z3/b0)
i_req = i_track + i_dist
i_cmd = slew_limit(amplitude_limit(i_req))
```

制御用単位はrad、rad/s、 rad/s^2 、A、秒に統一する。帯域 w_c/w_o はrad/s、Hzとの混同をテストする。 b_0 の符号は実機確認事項とし、値・符号の未検証状態で高電流を許可しない。

主制御の観測は位置のみで成立させる。既存の速度フィードバックは診断比較に残す。生の位置差分をDへ直結しない。

5. C4：ESO・補償制限

既存の離散ESOゲインはZOH/current observerの係数なので、根拠なく連続時間Euler版へ差し替えない。制限後の電流を戻す既存の正しい構造を維持する。未送信電流を印加入力としない。同じ測定で重複補正しない。

外乱補償に上限・変化率上限を持たせ、誤差依存リークからの変更は独立比較可能にする。ESO更新を単純に飽和中ずっと凍結する方式は採らず、制限後入力とモデル妥当性に基づいて処理する。拘束・長時間飽和・進行不足は停止条件とする。

受信時刻での推定と制御時刻への予測を区別する。近似を採用する場合は条件と残留遅延を明記し、テストに遅延を含める。

6. C5：任意の200 Hz化と比較モード

100 Hz版で効果を確認後、設定でCyberGearのみphase 0/5の200 Hzへ変更可能にする。TIM6の1 msを変えず、RobStride phase 1/2/3と上位送信phase 9を維持する。

運動制御モードでのPD+同じqd/vd軌道を比較できる設計は許可するが、走行中に電流モードから自動切替しない。比較モードは停止状態で明示選択・readback・再初期化する。未検証の「最適ゲイン」を自動で作って投入しない。

7. テストと成果物

ホストテストでは軌道の連続性・制約、制御の符号と単位、電流制限、ESO係数、重複／古い測定、tick wrap、NaN/Inf、受信フレームのID/DLC/type分岐、TX失敗、停止再試行、モード誤認防止を検証する。添付の04_数式確認.pyは参考計算であり、実装テストの代わりにはしない。

1000 TIM6 tickで、通常100 Hz版ならCG100回、200 Hz版ならCG200回、各RobStride100回、上位送信100回となる帰帰テストを加える。故障状態のときは正常指令数ではなく状態機械の規定どおりになることを検証する。

PCでASan/UBSanと警告を利用できる場合は実行する。少なくとも新規の純C計算部に警告を残さない。-ffast-math等でfinite判定が無効化される設定を採用しない。既存ビルド全体を無断変更せず、必要なら対象ソースの設定と影響をレビューする。

未同定モデルによるシミュレーションを「実機で安定」と記載しない。b/b0の変化、遅延、飽和、摩擦、柔軟性に対する失敗領域も示す。

作業終了時には、変更ファイル一覧、各変更の目的、テストコマンドと結果、ビルド結果、実機未検証事項、人間が決める設定値、範囲外候補、ロールバック方法を出力する。未実行のテストを合格と書かない。

人間が実装前または通電前に埋める項目

項目	値／確認結果
許容位置範囲・外側保護範囲	未確定
許容電流の連続／短時間値	未確定。現行ソフト10 Aをそのまま定格と扱わない
最高速・加減速・jerk	未確定
実機b0の符号・代表2姿勢の範囲	未確定
搭載FW、モードreadback	未確認
内蔵通信タイムアウトの有無・動作	未確認
停止時の慣性回転・外部停止方法	未確認
原点設定で静止を待つ仕様変更	要承認
上位returnの2秒と完了判定の関係	仕様確認のみ。今回変更しない

範囲外の不具合候補・仕様確認メモ

基準：2a888ed1e31dda6a063dd62f70259de9111d552a / 2026-09-09

このメモは修正指示ではない。複数人開発のため、仕様に見える挙動を推測で変更しない。「確認済み」はソース上の構造が確認できたことを指し、実機事故や障害の再現を意味しない。

B01：全体Error_Handlerは駆動停止を行わない

優先度：高 / 分類：コード上の安全上の欠落候補。

場所：Core/Src/main.c、Error_Handler()、初期化失敗時の呼出し。[S2]

割込みを禁止して無限ループへ入るだけで、各モーターへ停止要求を出さない。途中まで有効化された他モーターが存在する経路では、最後の目標が残る可能性がある。外部安全回路やモーターFWのタイムアウトが対策しているかは不明。

今回、CyberGear専用の起動失敗ではCyberGear停止管理へ入れる。一方、全モーターの停止順序、外部停止、Error_Handlerの再設計は共同担当者の承認事項。

B02：フィードバック監視が全モーター共通

優先度：高 / 分類：監視仕様の確認。

場所：motor_check_feedback()、FDCAN3受信処理。[S2]

4台のどれかから既知のfeedbackを受けると、共通motor_last_feedback_msが更新される。したがって一部モーターのみの通信断は、この全体監視からは検出されない。CyberGearには個別監視があるが、同じ個別監視が他モーターにもあるとは確認できない。

3秒の全体無受信でMCUをresetする仕様にも注意。reset時のモーター駆動状態と、その後の初期化要求待ちを含む安全性は別途検討が必要。今回共通仕様は変更しない。

B03：他軸の周期送信失敗を呼出し側で処理していない

優先度：高 / 分類：コードで確認、影響は実機依存。

場所：HAL_TIM_PeriodElapsedCallback()。[S2]

RobStride位置指令、FDCAN1の角度返信で戻り値を使っていない。TX FIFO満杯やbus-offで送信が失敗しても、上位へそのまま有効な動作状態を示し得る。現行の瞬間詰まりの有無はログで確認する。

CyberGearのTX監視は追加するが、他軸の失敗方針まで一括変更しない。

B04：上位目標の失効・数値妥当性が未統一

優先度：高 / 分類：上位通信仕様の確認。

場所：FDCAN1 CANID=0x200 受信処理。[S2]

4 floatを受信してそのままtarget_angle[]へ代入する。上位目標の時刻管理や、全軸に対するfinite・可動域検証はこの処理にはない。通信断時の目標保持が意図した動作かを確認する必要がある。

CyberGear側は従来もfiniteとプロトコル範囲を確認している。今回追加するのはCyberGearの機械範囲・必要な専用失効処理まで。他軸の意味や上位パケット形式は変えない。

B05：受信値組立ての符号付き左シフト

優先度：中／分類：C言語上の未定義動作候補として確度が高い。

場所：u8_to_float()、u8_to_int()。[S2]

req[i*4] << 24 は、uint8_tがintへ整数拡張された後にシフトされる。32 bit int環境では、先頭byteが0x80以上のとき、正のintで表現できない結果への左シフトとなる。負のfloat等がこの条件を通る。

通常のコンパイラで意図通りに見えることは、言語上の正しさの保証ではない。別作業で、シフト前に各byteをuint32_tへcastすることを提案する。今回のCyberGear限定変更に乗せない。

B06：RobStride 03/05に共通のフィードバック換算

優先度：中／分類：要マニュアル照合、バグ未確定。

場所：Core/Src/robstride_app.c、robstride_parse_feedback()。[S7]

位置±12.57、速度±20、トルク±60をすべてのRobStrideへ同じように適用している。異なる型番・FWで同一レンジか、velocity_rps という名称と実際の単位が一致するかを確認したい。位置指令が動くことだけでは速度・トルクの監視値が正しいとは限らない。

メーカー資料との照合前に、推測でスケールを変更しない。

B07：初回0x500コマンドは基準値として消費する

優先度：中／分類：意図的仕様の可能性が高い。

場所：FDCAN1 MOTOR_INIT_CANID 分岐。[S2]

最初の有効なint32は比較基準になり、次の値変化で初期化要求を立てる。起動時に一度だけ送る上位実装では初期化されない。コメントから意図が読み取れるので、単独で「バグ」と判断しない。

上位担当者と、初期値、toggle方式、再起動時の再同期を確認する。今回変更しない。

B08：return中の2秒は到達確認ではない

優先度：中／分類：意図したシーケンス仕様か確認。

場所：motor_return_update()、MOTOR_RETURN_IGNORE_MS。[S2]

return要求後は目標値を固定し、2秒経過で新コマンド受付を再開する。モーターの実到達・低速度・誤差収束では判定していない。高速化しても、他軸が未到達かもしれない一方、十分早く着いても待ち続ける可能性がある。

上位の動作仕様に関わるので変更しない。改修後のCyberGearが他軸より先に動くことで衝突しないかは、人間が統合試験で確認する。

B09：割込み禁止中にRX FIFOを空になるまで処理する

優先度：中／分類：リアルタイム性の候補。

場所：motor_return_update() からFDCAN1受信コールバックの直接呼出し。[S2]

割込みを禁止した状態で、空になるまでFIFOを処理する。通常は短くても、継続受信時の実行時間の上限をコードから読み取りにくい。同優先度のFDCAN/TIM6 ISRも含め、制御周期に長い外れ値を作る可能性がある。[S4]

まずDWT等で実行時間を計測し、必要なら上限件数付き排出等を別PRで検討する。CyberGear改修では周期超過を検出して記録する。

B10：UARTの戻り値が実送信量を反映しない

優先度：低 / 分類：診断信頼性の問題候補。

場所：_write()。[S2]

HAL_UART_Transmit(..., 10) の結果にかかわらずlenを返す。115200 baudでは長文や割込みによる待ち時間でログ欠落があり得る。短い行にしている現状の対策はあるが、連続・完全ログの保証にはならない。

現在の周期制御は割込み側なので、これを直ちに「UARTで制御が停止する原因」とはしない。CyberGearログは非ブロッキングのリングバッファを優先する。

B11：共有CAN設定は欠陥と即断しない

優先度：観測 / 分類：設計意図の確認。[S2][S4]

AutoRetransmission=DISABLE は送信失敗率に影響するが、古い制御指令を延々再送しない意図もあり得る。無断で有効化しない。受信フィルタが広いことは負荷・ノイズ流入の検討点だが、必要な未知フレームの観測目的もあり得る。

FDCANのコントローラ設定がFD_BRSでも、モーター用TXヘッダがClassic CAN/BRS OFFであること自体は矛盾ではない。ボード間big-endian、モーターパラメータlittle-endian、モーション16 bit値big-endianという違いも直ちにバグではない。

境界項目：CyberGearホーミング

STOP後10 msで原点設定し、静止を待たない点はコードのコメントと一致する。平均エッジ位置の意味やゼロ時の残留回転は通常制御に影響するが、原点の仕様変更は担当者確認後とする。今回の必須境界対策は、原点変更後の新鮮なfeedback、状態リセット、通常制御開始時の残留速度確認である。[S2]

根拠

[S2][S4] のURLと関数一覧は 01_改修説明書.md の参照資料に記載。

[S7] RobStride実装： [ソースを開く](#)

このメモはコアの制御・通信関連ファイルを対象とした限定レビューであり、リポジトリ全体を監査したものではない。

数式検算の概要

独立Python検算：7テストメソッド合格、失敗0、エラー0。これは説明書の算術・式・例を検算した結果であり、対象Cコードを呼び出してはいません。

確認した事項

対象	結果・範囲
離散ESO	リークなしで、dt=2/5/10/20/50 msの特性多項式が三重極 $p=\exp(-\omega dt)$ と一致
現行公称モデル	ランプ誤差0.4 rad、極-1.0718/-14.9282 [1/s]、スルー反転の数値を確認
五次軌道	任意初期 $q/v/a$ から終端 $qf/0/0$ への4例の端点条件を確認
静止端点の制約式	90度、 $V=1.5$ 、 $A=3$ 、 $J=15$ の例で解析的な最大値と制約を確認
CANエンコード例	run_mode=3、iq_ref=1 A、STOPのID/byte規約を独立生成で確認
スケジューラ例	1000 tickでCG100/200回、他軸・上位各100回を例示コードで確認
入力条件	一部の非有限値・非正時間／制限値の拒否を確認

ESO特性多項式の係数差は最大 4.44e-16。丸め誤差の範囲です。

90度の参照時間は、現行ランプで 3.926991 秒、五次軌道の設計例で 1.963495 秒でした。後者の解析的最大速度は 1.500000 rad/s、最大加速度は 2.352337 rad/s²、最大jerkは 12.450347 rad/s³です。

未確認事項

一般の再計画問題で常に制約を満たすこと、改修C実装、実際のCANフレーム受理、遅延・摩擦・柔軟性を含むプラントでの安定性、実機の安全性・到達時間は確認していません。これらは説明書の実装後テスト・実機試験で検証します。

検算の再現には同梱 04_数式確認.py を使用します。全数値は 05_数式確認結果.json に保存しています。